

When Optimization Outruns Feedback

The Feedback Horizon Gap in AI Optimization

Gunnar Zarncke

August 2026

Abstract

AI systems increasingly improve by running fast generate–evaluate–update loops against formal verifiers, tests, reward models, benchmarks, simulations, or other evaluators. Some of these evaluators are close to the target of interest. Others are proxies whose important errors become visible only later. The word “feedback” hides a count: how many proxy-guided optimization updates occur before sufficiently independent target-level evidence becomes available. Call that count N_{proxy} . When updates are approximately periodic, $N_{\text{proxy}} \approx \tau_{\text{T}}/\tau_{\text{P}}$, where τ_{P} is the proxy-feedback latency and τ_{T} the target-evidence latency. Target loss decomposes exactly as

$$L_{\text{T}}(\theta_{N_{\text{proxy}}}) - L_{\text{T}}(\theta_0) = \sum_{i=0}^{N_{\text{proxy}}-1} [L_{\text{T}}(\theta_{i+1}) - L_{\text{T}}(\theta_i)].$$

Systematic proxy error can therefore accumulate for N_{proxy} selected updates before independent evidence can correct the process. Reward-model overoptimization shows the failure mode in AI. Formal theorem proving and executable code show the opposite case, where rapid verification stays tied to the target. Weather forecasting, autonomous laboratories, and surrogate clinical endpoints sit in between: optimization and target evidence run on different clocks. As optimization accelerates, the operative variable is how many consequential updates occur between independent target checks, not merely how quickly an evaluator returns a score.

1 Introduction

Problems with clear, cheap verification are unusually friendly to optimization. A compiler rejects invalid syntax. A unit test can reject a broken implementation. Lean rejects an invalid formal proof. A game engine returns an exact win or loss. In each case, candidate generation can be coupled to a short feedback loop whose output is strongly connected to the local target.

Many important problems do not have this structure. A reward model can score a response immediately while human preference outside the reward model’s training distribution is only sampled occasionally. A weather model can produce a ten-day forecast in under a minute while the ten-day atmosphere still takes ten days to occur (Lam et al. 2023). A drug may improve a surrogate endpoint months or years before its effect on survival or quality of life is known, and confirmatory evidence can arrive years after an accelerated approval (Gyawali, Hey, and Kesselheim 2019; Liu, Kesselheim, and Cliff 2024). In these cases, “feedback speed” is not one number.

Optimization runs on one loop; independent target evidence arrives on another. The gap is measured in *selected optimization updates* between independent target checks, not wall-clock latency alone.

Let N_{proxy} denote that count. If proxy updates occur at roughly regular intervals τ_P and informative target evidence arrives every τ_T , then

$$N_{\text{proxy}} \approx \left\lfloor \frac{\tau_T}{\tau_P} \right\rfloor. \quad (1)$$

Parallel search, batching, asynchronous evaluation, and variable update sizes can make wall-clock time a poor proxy. Count updates directly.

Large N_{proxy} can coexist with safety. When the proxy evaluator matches the target criterion, millions of updates may do no harm. AlphaProof uses Lean-verified outcomes as grounded feedback and learns from millions of formally checked problems (Hubert et al. 2025). The failure mode appears when repeated selection acts on an imperfect proxy whose errors stay hidden across many updates before an independent check.

Three questions often get conflated:

- (i) How fast can candidates be generated?
- (ii) How fast can the signal used for optimization be computed?
- (iii) How fast can sufficiently independent evidence about the target become available?

AI can compress the first two by orders of magnitude without necessarily compressing the third.

2 Timescales and Accumulation

2.1 Four timescales

We distinguish four latencies:

$$\tau_G = \text{candidate-generation latency}, \quad (2)$$

$$\tau_P = \text{latency of feedback actually used for optimization}, \quad (3)$$

$$\tau_T = \text{latency until informative target-level evidence is available}, \quad (4)$$

$$\tau_C = \text{latency until that evidence can materially correct the optimizer or system}. \quad (5)$$

The target need not be metaphysical “ground truth.” It is the outcome we actually care about for the analysis. The key requirement is that target evidence have at least partially independent failure modes from the proxy used to select updates. For a reward model, a fresh human evaluation can serve as target evidence. For a ten-day weather forecast, the realized atmosphere does. For an accelerated drug approval based on tumor response, overall survival or quality of life can provide a more target-relevant endpoint.

Usually, but not always,

$$\tau_G \leq \tau_P \leq \tau_T \leq \tau_C.$$

A slow proxy evaluator can have $\tau_P > \tau_T$, and target evidence may exist before an institution notices or acts on it.

2.2 The feedback horizon

Let θ_i be the system state after the i -th proxy-guided update following an independent target check. Define

$$N_{\text{proxy}} = \text{number of proxy-guided updates before the next independent target check}. \quad (6)$$

Equation (1) follows only for approximately regular sequential updates.

The distinction matters for AI systems because a million candidates may be generated and filtered in parallel during one hour. Wall-clock latency says “one hour.” The relevant exposure can be closer to one million selected comparisons or update opportunities.

2.3 Accumulation identity

Let $L_T(\theta)$ denote target loss, with lower values better. Define the target effect of proxy-guided update i as

$$\Delta_i = L_T(\theta_{i+1}) - L_T(\theta_i).$$

Then exactly,

$$\boxed{L_T(\theta_{N_{\text{proxy}}}) - L_T(\theta_0) = \sum_{i=0}^{N_{\text{proxy}}-1} \Delta_i.} \quad (7)$$

The identity holds regardless of the optimizer, the proxy, linear response, smoothness, stationarity, or whether the target is observable during optimization.

Equation (7) separates how long error can accumulate from whether it does. A perfect evaluator yields $\Delta_i \leq 0$ under selection; a large N_{proxy} then helps. An exploitable proxy yields positive Δ_i that can stack for many selected updates before independent evidence arrives.

Under the additional assumption that Δ_i are identically distributed with mean μ ,

$$\mathbb{E}[L_T(\theta_{N_{\text{proxy}}}) - L_T(\theta_0)] = N_{\text{proxy}}\mu. \quad (8)$$

Systematic per-update target drift then accumulates linearly in the number of unchecked updates.

If $\mu = 0$, $\text{Var}(\Delta_i) = \sigma^2$, and the errors are independent,

$$\text{SD}\left(\sum_i \Delta_i\right) = \sigma\sqrt{N_{\text{proxy}}}. \quad (9)$$

More generally, for weakly stationary update effects with autocovariance $\gamma_h = \text{Cov}(\Delta_i, \Delta_{i+h})$,

$$\text{Var}\left(\sum_{i=0}^{N_{\text{proxy}}-1} \Delta_i\right) = N_{\text{proxy}}\sigma^2 + 2 \sum_{h=1}^{N_{\text{proxy}}-1} (N_{\text{proxy}} - h)\gamma_h. \quad (10)$$

Persistent positive correlation in proxy errors therefore makes repeated optimization more dangerous than independent noise. This is the regime one would expect when an optimizer repeatedly exploits the same evaluator blind spot.

2.4 Reading N_{proxy} in context

A theorem checker and a learned reward model can share the same N_{proxy} and differ sharply in reliability. Interpretation turns on:

- **Fidelity:** how informative is proxy improvement about target improvement?
- **Independence:** are later target checks genuinely independent, or do they reuse the same blind spots?
- **Reversibility:** can proxy-driven changes be cheaply undone?
- **Decomposability:** can target failure be detected locally before the full target horizon?
- **Consequence per update:** how far can one accepted update move the deployed system?

3 Direct Evidence: Reward-Model Overoptimization

Reward-model optimization instantiates Equation (7). In standard RLHF, a learned reward model predicts human preferences and is then optimized by a policy. The reward model is cheaper to query than humans and therefore becomes the fast proxy loop.

Gao, Schulman, and Hilton constructed a controlled setting in which a fixed “gold” reward model supplied labels used to train a weaker proxy reward model (Gao, Schulman, and Hilton 2022). They then optimized the policy against the proxy using reinforcement learning or best-of- n sampling. As proxy optimization increased, gold-model performance eventually degraded. What matters empirically is the separation:

$$\text{proxy score } \uparrow \quad \text{while} \quad \text{independent target proxy } \downarrow.$$

The optimizer had enough search power to discover outputs that scored well according to the learned evaluator but worse according to the independently held reference evaluator.

Rafailov et al. extended the observation to direct alignment algorithms such as DPO, which do not expose a separate learned reward model during optimization (Rafailov et al. 2024). They found analogous deterioration at higher optimization budgets and reported that degradation could occur before one full epoch of the preference dataset was completed. Overoptimization is not confined to explicitly training against a separate scalar reward network. Repeated selection against a finite preference signal can move the policy into regions where the optimization signal ceases to track the broader evaluation criterion.

More recent work interprets the same phenomenon partly as distribution shift. Ackermann, Ishida, and Sugiyama show that as the optimized policy moves away from the reward model’s training distribution, the reward-model parameter estimate and resulting policy gradient can become inconsistent; an off-policy correction improves final policy quality in their experiments (Ackermann, Ishida, and Sugiyama 2025). Iterated RLHF can reduce overoptimization when new reward models better approximate the target distribution, but early overoptimization is not always fully recovered (Wolf, Kirk, and Musolesi 2025).

In the present notation, reward-model training creates a fast evaluator P . Policy optimization can perform many updates before fresh human labels or otherwise independent target checks update P . Wall-clock τ_T varies across RLHF pipelines. The qualitative pattern is stable: continued optimization against a fixed imperfect evaluator increases accumulated target error before the evaluator is corrected.

Goodhart’s law names proxy–target divergence. The feedback horizon counts how much *selection* can run before that divergence meets an independent check.

4 Large N_{proxy} with Strong Verifiers

Formal theorem proving and executable software carry large N_{proxy} without the reward-model failure mode.

4.1 Formal theorem proving

Formal proof assistants sharply separate candidate generation from verification. Lean can reject an invalid tactic or proof according to a fixed formal semantics. AlphaProof turns this into an RL environment: tactics are actions, Lean states are observations, and Lean-verified proof, disproof, or timeout outcomes provide grounded feedback (Hubert et al. 2025). Its training uses millions

of auto-formalized problems, and its test-time RL can generate and learn from millions of related variants. At the 2024 IMO, the AlphaProof-centered system solved three non-geometry problems, with difficult solutions requiring two to three days of test-time RL (Hubert et al. 2025).

This is a large- N_{proxy} regime by construction. The system makes enormous numbers of search and learning moves before any human reads the final proof. The local target, formal validity in Lean, is checked inside the fast loop.

Formal verification checks the formal statement. If the formalization misses the intended natural-language problem, the verifier can be perfectly reliable about the wrong target. AlphaProof therefore splits into two clocks:

proof validity: $\tau_P \approx \tau_T$ once formalized,
 faithful formalization: τ_T may require separate human or semantic checking.

4.2 Executable software

Execution feedback gives code generation a similarly short local loop. Chen et al. showed that LLM self-debugging using execution results and feedback messages improves code-generation performance by up to 12 percentage points on benchmarks with unit tests and can match or exceed baselines generating over ten times as many candidate programs (Chen et al. 2023). Here the test harness supplies dense, actionable information about some classes of failure.

Human development benefits from the same structure. In a randomized experiment with 95 professional developers implementing the same JavaScript HTTP server, developers using GitHub Copilot completed the task in an average of 1 hour 11 minutes versus 2 hours 41 minutes without Copilot, a 55.8% reduction in completion time (Peng et al. 2023). The submissions were evaluated with a test suite.

Software often mixes both regimes in one artifact:

fast trusted feedback for covered properties + slow or missing feedback for uncovered properties.

The same codebase can pass unit tests quickly while security and maintainability failures surface only under deployment or audit (Fu et al. 2023).

5 Why AI Changes the Gap

AI changes the feedback geometry in a specific way. It can reduce τ_G , reduce some forms of τ_P , and increase parallel search capacity without changing the rate at which external reality produces independent evidence.

GraphCast is a clean illustration. A ten-day global forecast can be generated in under one minute, but the atmosphere cannot reveal the full ten-day error in under ten days (Lam et al. 2023). The compute-to-reality ratio has changed by orders of magnitude even though the physical target horizon has not.

Self-driving laboratories show a different response. Instead of merely accepting the physical delay, they attack it. A-Lab integrates planning, robotics, synthesis, characterization, and active learning in one loop, performing 353 experiments in 17 days (Szymanski et al. 2023). AlphaFlow uses in-situ measurements to turn intermediate physical states into feedback for subsequent experimental choices (Volk et al. 2023). These systems reduce the feedback horizon by building better measurement infrastructure around the physical process.

Clinical medicine compresses decisions onto surrogate endpoints because patient-centered endpoints take much longer to observe. Faster surrogate feedback pays off only when the surrogate tracks

survival, function, or quality of life. Reviews of accelerated oncology approvals find many drugs approved on surrogate evidence that never show patient-centered gains, or remain uncertain years later (Gyawali, Hey, and Kesselheim 2019; Liu, Kesselheim, and Cliff 2024). Faster optimization has not shortened the biological target horizon; it has moved decisions onto an earlier, imperfect signal.

Behavioral evals, preference models, constitutional graders, and automated red teams supply rapid alignment signals. Robustness under capability growth, distribution shift, strategic behavior, or long deployment can still require target evidence on a much longer clock. A one-second score can proxy a one-year claim.

6 Scope and Limits

6.1 Perfect or sufficient proxies

When proxy feedback matches the target, a long horizon gap costs nothing. Formal theorem checking on a fixed statement is the standard example.

6.2 Search difficulty

Formal proofs and NP-hard search problems can be cheap to verify and expensive to find. Verification latency and search hardness are separate axes.

6.3 Plural targets

For social welfare, governance, or alignment, there may be no single scalar L_T . “Target evidence” then means an independent check on whichever outcome dimension matters for the decision; N_{proxy} depends on that choice.

6.4 Parallelism

The ratio τ_T/τ_P understates exposure when many candidates run in parallel and overstates it when updates are tiny or reversible. Count consequential proxy-guided updates between target checks directly.

7 Discussion

The operative distinction is not easy versus hard problems. It is whether optimization runs inside a loop whose evaluator is both fast and tied to the target.

That split explains rapid AI progress in formal mathematics, games, and test-covered software. Candidate generation is cheap, and these domains already ship reliable selectors. AlphaProof runs large-scale RL because Lean supplies grounded proof feedback (Hubert et al. 2025). Code models self-debug because execution makes some failures legible (Chen et al. 2023). The verifier need not know how the candidate was produced.

The same structure inverts when the evaluator is imperfect. Gao et al. show proxy reward rising while an independent evaluation falls under continued optimization (Gao, Schulman, and Hilton 2022). Per-update target errors sum for as many updates as the system takes before correction. Faster optimization raises that count unless independent evaluation keeps pace.

The local question behind “capabilities may outrun alignment” is concrete:

How many consequential optimization updates can occur before an evaluator with sufficiently independent failure modes gets to reject the direction of travel?

Sometimes the answer is one: a failed Lean proof rejects immediately. Sometimes it is millions: a fixed reward model guides a long RL run before fresh human preference data arrives. Sometimes physics sets the clock: a ten-day forecast needs a ten-day world. Sometimes biology or institutions do: a surrogate endpoint can precede patient-centered clinical evidence by years.

Evaluation research shortens the target-relevant horizon or it does not. A faster proxy score without faster independent target evidence increases optimization speed without increasing correction speed. How to rebuild that evidence channel, and how the available reconstructions differ across domains, remains future work.

8 Conclusion

The Feedback Horizon Gap is the count

N_{proxy} = proxy-guided optimization updates between independent target checks.

For regular sequential systems, $N_{\text{proxy}} \approx \tau_T / \tau_P$. Target drift over that window is

$$L_T(\theta_{N_{\text{proxy}}}) - L_T(\theta_0) = \sum_i \Delta_i.$$

Target-safe proxy updates make large N_{proxy} cheap. Systematic or correlated proxy error accumulates until independent evidence arrives.

Reward-model overoptimization shows the failure mode in AI. Formal theorem proving and execution-tested software show large throughput with strong verifiers. Weather forecasting, autonomous laboratories, and clinical surrogate endpoints show optimization and target evidence on mismatched clocks.

As optimization accelerates, reliability tracks how many consequential updates get selected before sufficiently independent target evidence arrives, not how quickly a proxy returns a score. That count is readable in many AI systems today; cross-domain prediction beyond the reward-overoptimization cases remains to be tested.

References

- Ackermann, J., T. Ishida, and M. Sugiyama (2025). *Off-Policy Corrected Reward Modeling for Reinforcement Learning from Human Feedback*. arXiv: 2507.15507 [cs.LG]. URL: <https://arxiv.org/abs/2507.15507>.
- Chen, X., M. Lin, N. Schärli, and D. Zhou (2023). *Teaching Large Language Models to Self-Debug*. arXiv: 2304.05128 [cs.CL]. URL: <https://arxiv.org/abs/2304.05128>.
- Fu, Y., P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, and J. Chen (2023). *Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study*. arXiv: 2310.02059 [cs.SE]. URL: <https://arxiv.org/abs/2310.02059>.
- Gao, L., J. Schulman, and J. Hilton (2022). *Scaling Laws for Reward Model Overoptimization*. arXiv: 2210.10760 [cs.LG]. URL: <https://arxiv.org/abs/2210.10760>.
- Gyawali, B., S. P. Hey, and A. S. Kesselheim (2019). “Assessment of the Clinical Benefit of Cancer Drugs Receiving Accelerated Approval”. In: *JAMA Internal Medicine* 179.7, pp. 906–913. DOI: [10.1001/jamainternmed.2019.0462](https://doi.org/10.1001/jamainternmed.2019.0462).

- Hubert, T., R. Mehta, L. Sartran, M. Z. Horváth, G. Žužić, E. Wieser, A. Huang, et al. (2025). “Olympiad-Level Formal Mathematical Reasoning with Reinforcement Learning”. In: *Nature* 651, pp. 607–613. DOI: [10.1038/s41586-025-09833-y](https://doi.org/10.1038/s41586-025-09833-y).
- Lam, R., A. Sanchez-Gonzalez, M. Willson, P. Wirsberger, M. Fortunato, F. Alet, S. Ravuri, et al. (2023). “Learning Skillful Medium-Range Global Weather Forecasting”. In: *Science* 382.6677, pp. 1416–1421. DOI: [10.1126/science.adi2336](https://doi.org/10.1126/science.adi2336).
- Liu, I. T. T., A. S. Kesselheim, and E. R. S. Cliff (2024). “Clinical Benefit and Regulatory Outcomes of Cancer Drugs Receiving Accelerated Approval”. In: *JAMA* 331.17, pp. 1471–1479. DOI: [10.1001/jama.2024.2396](https://doi.org/10.1001/jama.2024.2396).
- Peng, S., E. Kalliamvakou, P. Cihon, and M. Demirel (2023). *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot*. arXiv: [2302.06590](https://arxiv.org/abs/2302.06590) [cs.SE]. URL: <https://arxiv.org/abs/2302.06590>.
- Rafailov, R., Y. Chittepudi, R. Park, H. Sikchi, J. Hejna, B. Knox, C. Finn, and S. Niekum (2024). *Scaling Laws for Reward Model Overoptimization in Direct Alignment Algorithms*. arXiv: [2406.02900](https://arxiv.org/abs/2406.02900) [cs.LG]. URL: <https://arxiv.org/abs/2406.02900>.
- Szymanski, N. J., B. Rendy, Y. Fei, R. E. Kumar, T. He, D. Milsted, M. J. McDermott, et al. (2023). “An Autonomous Laboratory for the Accelerated Synthesis of Inorganic Materials”. In: *Nature* 624, pp. 86–91. DOI: [10.1038/s41586-023-06734-w](https://doi.org/10.1038/s41586-023-06734-w).
- Volk, A. A., R. W. Epps, D. T. Yonemoto, B. S. Masters, F. N. Castellano, K. G. Reyes, and M. Abolhasani (2023). “AlphaFlow: Autonomous Discovery and Optimization of Multi-Step Chemistry Using a Self-Driven Fluidic Lab Guided by Reinforcement Learning”. In: *Nature Communications* 14, p. 1403. DOI: [10.1038/s41467-023-37139-y](https://doi.org/10.1038/s41467-023-37139-y).
- Wolf, L., R. Kirk, and M. Musolesi (2025). *Reward Model Overoptimisation in Iterated RLHF*. arXiv: [2505.18126](https://arxiv.org/abs/2505.18126) [cs.LG]. URL: <https://arxiv.org/abs/2505.18126>.